Practical - 13

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

SEMESTER: 3RD, 2026-27

ID: 25DCE110

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory

Answers to key questions

1. Do PHP endpoints return valid JSON responses?

Yes. PHP endpoints return JSON data using json_encode() and set the Content-Type header to application/json.

2. Are GET and POST requests handled properly?

Yes. GET retrieves records, while POST submits data to create records. Update and delete operations can use POST with an action field or other explicitly supported HTTP methods.

3. Are loading, success, and error states shown on the frontend?

Yes. JavaScript displays loading messages while requests are running and success or error messages after the response.

4. Is input validation performed before database operations?

Yes. PHP validates the received data on the server before executing database queries.

Implementation/ Output/Result

Tags:

Demo code:

```
<?php
header("Content-Type: application/json; charset=utf-8");

require_once __DIR__ . "/db_mysqli.php";

function respond($status, $data) {
    http_response_code($status);
    echo json_encode($data);
    exit;
}

if ($_SERVER["REQUEST_METHOD"] === "GET") {
    $result = $conn->query(
        "SELECT id, name, email, course FROM students ORDER BY id DESC"
    );

    $students = $result->fetch_all(MYSQLI_ASSOC);
    respond(200, $students);
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $data = json_decode(file_get_contents("php://input"), true);
```

```
if (
    !is_array($data) ||
    !isset($data["name"], $data["email"], $data["course"]) ||
    !is_string($data["name"]) ||
    !is_string($data["email"]) ||
    !is_string($data["course"]) ||
    trim($data["name"]) === "" ||
    !filter_var($data["email"], FILTER_VALIDATE_EMAIL) ||
    trim($data["course"]) === ""
) {
    respond(400, array("error" => "Invalid student details."));
}

$name = trim($data["name"]);
$email = trim($data["email"]);
$course = trim($data["course"]);

$stmt = $conn->prepare(
    "INSERT INTO students (name, email, course)
     VALUES (?, ?, ?)"
);

$stmt->bind_param("sss", $name, $email, $course);
$stmt->execute();
```

```
    respond(201, array(
        "success" => true,
        "message" => "Student added successfully."
    ));
}

respond(405, array("error" => "Method not allowed."));
?>
```

This example covers GET and POST only. Full Practical 13 also requires update and delete endpoints.

- Created PHP endpoints to communicate with the database.
- Returned data in JSON format.
- Used GET to retrieve student records.
- Used POST to add student records.
- Used JavaScript Fetch API to send asynchronous requests.
- Displayed loading, success, and error messages.
- Validated input before database operations.
- Used prepared statements to improve query safety.

GitHub Repository Link:

https://github.com/kanan2007s-sys/Web-development-/tree/main

Learning Outcomes:

Learned to integrate JavaScript Fetch API with PHP JSON endpoints for asynchronous database operations without full-page reloads.

Faculty Remarks: